

```

import pandas as pd          # For handling data (DataFrame)
import seaborn as sns        # For visualizations and Planets dataset
import numpy as np           # For numerical operations
# import matplotlib.pyplot as plt # For plots
# import math

# from pandas import plotting

# import statsmodels.formula.api as smf
# from sklearn.linear_model import LinearRegression
# from sklearn.model_selection import train_test_split
# from sklearn.metrics import mean_squared_error, r2_score

```

```

Drivers_df = pd.read_csv("https://raw.githubusercontent.com/Alissa-Ousp/Drivers_df.head()

```

	Report_Number	Agency_Name	ACRS_Report_Type	Crash_Date_Time	Route
0	MCP293200B1	MONTGOMERY	Injury Crash	5/5/26 22:43	M (State
1	MCP293200B1	MONTGOMERY	Injury Crash	5/5/26 22:43	M (State
2	DD5631006M	ROCKVILLE	Property Damage Crash	5/5/26 21:43	M (State
3	DD5631006M	ROCKVILLE	Property Damage Crash	5/5/26 21:43	M (State
4	MCP2737006P	MONTGOMERY	Property Damage Crash	5/5/26 21:30	

5 rows x 33 columns

```
# Upload Raw file from GitHub
```

```
# Skip column because many contain 2 values instead of 1
```

```
Incidents_df = pd.read_csv("https://raw.githubusercontent.com/Alissa-Ou")
Incidents_df.head()
```

	Report_Number	Hit_Run	Lane_Direction	Lane_Type	Road_Grade	At Fa
0	MCP293200B1	No	Southbound	Lane 2	Level	DRIV
1	DD5631006M	No	Eastbound	Lane 2	Downhill	UNKNC
2	MCP2737006P	No	Northbound, Westbound	Lane 1	Level	DRIV
3	MCP3386004J	Yes	Westbound	Unknown	Level	UNKNC
4	MCP3232007N	No	Northbound, Westbound	Lane 1	Level	DRIV

```
Drivers_df.info()
```

[Show hidden output](#)

```
Incidents_df.info()
```

[Show hidden output](#)

```
# Use left merge to keep all rows of Drivers_df, discard rows from Inc
#that don't have a match in the key (Report_Number) column
```

```
merged_df = pd.merge(
    Drivers_df, Incidents_df, how = "left", on = ["Report_Number"]
)
#merged_df.head()
```

```
#merged_df.info()
```

```
merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].apply(lambda x:
```

```
merged_df[merged_df["Vehicle_Year"] < 1930].count() # check if it worl
```

[Show hidden output](#)

```
merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].apply(lambda x:  
merged_df[merged_df["Vehicle_Year"] > 2026].count()
```

[Show hidden output](#)

```
merged_df.describe()
```

[Show hidden output](#)

```
merged_df.nunique()
```

[Show hidden output](#)

```
unique_agency_name = merged_df["Agency_Name"].unique()  
print(unique_agency_name)
```

```
['MONTGOMERY' 'ROCKVILLE' 'TAKOMA' 'GAITHERSBURG' 'MCPARK'  
 'Montgomery County Police' 'Rockville Police Departme'  
 'Gaithersburg Police Depart' 'Maryland-National Capital'  
 'Takoma Park Police Depart']
```

```
print(merged_df["ACRS_Report_Type"].unique())
```

```
['Injury Crash' 'Property Damage Crash' 'Fatal Crash']
```

```
column_list = merged_df.columns.tolist()  
print(column_list)
```

```
['Report_Number', 'Agency_Name', 'ACRS_Report_Type', 'Crash_Date_Time']
```

```
# run only once
for i in column_list:
    print(f"{i} : \n{merged_df[i].unique()}\n\n")
```

[Show hidden output](#)

```
column_list_abridged = ["Agency_Name", "ACRS_Report_Type", "Crash_Date"]
```

```
merged_cleaned_df = merged_df[column_list_abridged].copy()
merged_cleaned_df.head()
```

[Show hidden output](#)

```
# For all cells in merged_cleaned_df:
# if value in cell == "unkonwn" or "UNKNOWN" or "other" or "OTHER"
# or "Not Applicable" or "NOT APPLICABLE", then set value to nan
values_to_replace = ["unkonwn", "UNKNOWN", "Unknown", "other", "OTHER"]

for col in merged_cleaned_df.columns:
    if merged_cleaned_df[col].dtype == "object": # Only apply to object
        merged_cleaned_df[col] = merged_cleaned_df[col].replace(values_to_replace, np.nan)

display(merged_cleaned_df.head())
```

[Show hidden output](#)

```
# merge agency names
merged_cleaned_df["Agency_Name"] = merged_cleaned_df["Agency_Name"].replace({
    "MONTGOMERY" : "Montgomery County Police",
    "Rockville Police Departme" : "Rockville Police Dept",
    "ROCKVILLE" : "Rockville Police Dept",
    "Takoma Park Police Depart" : "Takoma Park Police Dept",
    "TAKOMA" : "Takoma Park Police Dept",
    "Gaithersburg Police Depar" : "Gaithersburg Police Dept",
    "GAITHERSBURG" : "Gaithersburg Police Dept",
    "Maryland-National Capital" : "MD-Natl Capital Park Pol",
    "MCPARK" : "MoCo Park Police"
})
display(merged_cleaned_df["Agency_Name"].unique())
```

```
array(['Montgomery County Police', 'Rockville Police Dept',
      'Takoma Park Police Dept', 'Gaithersburg Police Dept',
      'MoCo Park Police', 'MD-Natl Capital Park Pol'], dtype=object)
```

```

for col in merged_cleaned_df.select_dtypes(include='object').columns:
    unique_values = merged_cleaned_df[col].dropna().unique()

    replacement_map = {}
    grouped_values = {}

    # Group values by their lowercase form
    for value in unique_values:
        lower_value = value.lower()
        if lower_value not in grouped_values:
            grouped_values[lower_value] = []
        grouped_values[lower_value].append(value)

    # Create replacement map: all-caps -> first non-all-caps variant
    for lower_key, variants in grouped_values.items():
        all_caps_variants = [v for v in variants if v.isupper()]
        non_all_caps_variants = [v for v in variants if not v.isupper()]

        if non_all_caps_variants and all_caps_variants:
            # Choose the first non-all-caps variant as the target
            target_variant = non_all_caps_variants[0]
            for ac_variant in all_caps_variants:
                if ac_variant != target_variant: # Ensure no self-rep
                    replacement_map[ac_variant] = target_variant

    if replacement_map: # Only apply replace if there are actual repla
        merged_cleaned_df[col] = merged_cleaned_df[col].replace(replac
        print(f"Standardized capitalization for column: {col}")

# Display unique values for a few columns to verify the cleaning
print("\nUnique values after capitalization standardization:")
display(merged_cleaned_df["Agency_Name"].unique())
display(merged_cleaned_df["ACRS_Report_Type"].unique())
display(merged_cleaned_df["Weather"].unique())

```

[Show hidden output](#)

Compare values in each column. If a value written in all caps has a matching value that is written in mixed case or lower case, then convert all data written in all caps into the matching version that is written in mixed case or lower case.

```

classifiable_columns = ["Agency_Name", "ACRS_Report_Type", "Crash_Date

```

```
for i in classifiable_columns:
    print(f"{i} : \n{merged_cleaned_df[i].unique()}\n\n")
```

```
Agency_Name :
['Montgomery County Police' 'Rockville Police Dept'
 'Takoma Park Police Dept' 'Gaithersburg Police Dept' 'MoCo Park Poli
 'MD-Natl Capital Park Pol']
```

```
ACRS_Report_Type :
['Injury Crash' 'Property Damage Crash' 'Fatal Crash']
```

```
Crash_Date_Time :
['5/5/26 22:43' '5/5/26 21:43' '5/5/26 21:30' ... '1/1/15 1:14'
 '1/1/15 1:10' '1/1/15 0:30']
```

```
Route_Type :
['Maryland (State) Route' 'County Route' 'Spur' nan 'US (State)'
 'Local Route' 'Municipality Route' 'Other Public Roadway' 'Bicycle R
 'Ramp' 'Crossover' 'Interstate (State)' 'Private Route'
 'Government Route' 'Service Road' 'County' 'Maryland (State)'
 'Municipality' 'Government']
```

```
Related_Non_Motorist :
[nan 'Cyclist (non-electric)' 'Pedestrian' 'Cyclist (Electric)'
 'Occupant Of a Non-Motor Vehicle Transportation Device'
 'Scooter (non-Electric)' 'Scooter (electric)'
 'Other Pedestrian (person in a building, skater, personal conveyance
 'Wheelchair (electric)' 'Unknown Type Of Non-Motorist'
 'Occupant of Motor Vehicle Not in Transport'
 'Occupant of Motor Vehicle Not in Transport, Pedestrian'
 'Pedestrian, Wheelchair (non-electric)' 'Unknown, Wheelchair (electr
 'Wheelchair (non-electric)'
 'Other Pedestrian (person in a building, skater, personal conveyance
 'BICYCLIST' 'MACHINE OPERATOR/RIDER' 'OTHER PEDALCYCLIST'
 'OTHER, PEDESTRIAN' 'OTHER CONVEYANCE' 'OTHER CONVEYANCE, PEDESTRIAN
 'BICYCLIST, OTHER' 'BICYCLIST, PEDESTRIAN' 'IN ANIMAL-DRAWN VEH'
 'OTHER, OTHER CONVEYANCE']
```

```
Collision_Type :
['Front to Front' 'Front to Rear' 'Angle' 'Single Vehicle'
 'Sideswipe, Opposite Direction' nan 'Rear To Side' 'Rear To Rear'
 'Sideswipe, Same Direction' 'OPPOSITE DIRECTION SIDESWIPE'
 'HEAD ON LEFT TURN' 'STRAIGHT MOVEMENT ANGLE' 'SAME DIR REAR END'
 'HEAD ON' 'SAME DIRECTION SIDESWIPE' 'SAME DIRECTION RIGHT TURN']
```

```
'SAME DIRECTION LEFT TURN' 'ANGLE MEETS LEFT TURN'
'SAME DIR BOTH LEFT TURN' 'SAME DIR REND LEFT TURN'
'ANGLE MEETS RIGHT TURN' 'OPPOSITE DIR BOTH LEFT TURN'
'ANGLE MEETS LEFT HEAD ON' 'SAME DIR REND RIGHT TURN']
```

Weather :

```
['Clear' 'Rain' 'Cloudy' nan 'Fog, Smog, Smoke'
'Freezing Rain Or Freezing Drizzle' 'Sleet Or Hail' 'Snow' 'Blowing
'Severe Crosswinds' 'Blowing Sand, Soil, Dirt' 'RAINING' 'FOGGY' 'SL
'WINTRY MIX' 'SEVERE WINDS']
```

merged_cleaned_df.nunique()

	0
Agency_Name	6
ACRS_Report_Type	3
Crash_Date_Time	117933
Route_Type	18
Related_Non_Motorist	25
Collision_Type	23
Weather	15
Surface_Condition	11
Ambient_Light	9
Traffic_Control	28
Driver_Substance_Use	18
Non_Motorist_Substance_Use	25
Driver_At_Fault	2
Injury_Severity	5
Circumstance	854
Driver_Distracton	21
Drivers_License_State	81
Vehicle_ID	244070

Vehicle_ID	214213
Vehicle_Damage	6
Veh_1st_Impact_Loc	30
Veh_Body_Type	50
Vehicle_Movement	25
Veh_Going_Dir	9
Speed_Limit	16
Driverless_Vehicle	1
Parked_Vehicle	2
Vehicle_Year	72
Vehicle_Make	1963
Veh_Model	7191
Hit_Run	2
Lane_Type	197

```
merged_cleaned_df.to_csv("crash_data_cleaned_full")
```

Some categorical variables need to be cleaned up severely.

This is for another time.

- Related_Non_Motorist
- Lane_Type
- Road_Alignment
- Road_Division : caseblind, Drivers_License_State : nan = "XX", "MX-MEX", "MX-MX-ROO", "MX-GRO" = Mex, CA-ON, CA-QC, ON, QC, BC = Can

caseblind,

To be cleaned up now:

- "Agency_Name"
- "Intersection_Type"
- Lane_Type to Parking_Lot Y/N/nan
check against off road

- Vehicle_Damage: caseblind, "unkonwn" = "other" = nan

Veh_1st_Impact_Loc : "ROOF TOP" = "roof" = "top"; caseblind; OCLOCK = O Clock to O"Clock

- Driver_Distracton: Unknown = UNKNOWN = "NO DRIVER PRESENT" nan
- "Collision_Type" "other" = "unknown" = nan
- Veh_Body_Type : caseblind, unknown = other,

Vehicle_Movement : unknown = nan

- First_Harmful_Event : make caseblind, "Not Applicable" = nan
- Second_Harmful_Event : make caseblind, "Not Applicable" = nan
- Junction : "Non-Junction" = "NON INTERSECTION" = "UNKNOWN" = nan
- Intersection_Type : "UNKNOWN" = nan ;
"Roundabout/Traffic Circle" = "ROUNDAABOUT" = "TRAFFIC CIRCLE"
- Road Alignment: "OTHER" "UNKNOWN" = nan; make caseblind
- Road_Condition: "Unknown" nan; caseblind

Not useful or meaning unclear. Discard:

- Road_Grade
- Lane_Direction

Start coding or [generate](#) with AI.

Categorical Data:

ACRS_Report_Type : ['Injury Crash' 'Property Damage Crash' 'Fatal Crash']

Date/Time:

Date_Time

✓ Log of changes made to datasets:

Drivers:

- added underscores in column names to remove spaces
- abbreviated some column names
- deleted "location" column (already have latitude, longitude)
- deleted Local Case Number (Report Number is already identifier)

Incidents:

- added underscores in column names to remove spaces
- abbreviated some column names
- deleted "direction", "distance", "distance unit columns" due to contradiction of definitions in sources
- deleted Municipality (Blank)
- Number of Lanes (many contain 2 values)

Merged:

- Standardized Vehicle_Year column input years to four-digit e.g.
`merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(4, 2004)`

Start coding or [generate](#) with AI.

Future research:

It would be useful to interview representatives of the municipalities about how they classify and record specific details about crashes, such as drug and alcohol use. I would also like to know why the handbook lists many more fields in covered by the Automated Crash Reporting System (ACRS)

Maryland Department of State Police

<https://mdsp.maryland.gov/Pages/Dashboards/CrashDataDashboard.aspx>

Maryland Crash Data 2024-Present Map

Uses updated information, but that information is not shared with the public (only the preliminary information, which may be incomplete or contain errors for the reasons listed).

Age of driver

Include data about passengers: number of passengers,
general age

Note on Data

In trying to understand the meaning of certain fields, I may have found an error in the Incidents data description on

https://data.montgomerycountymd.gov/Public-Safety/Crash-Reporting-Incidents-Data/bhju-22kf/about_data

The "Direction" field is described as "Location - Direction from mile point."

The "Distance" field is described as "Location - Distance from mile point."

However, the Maryland Automated Crash Reporting System (ACRS) Field Reference Guide published on

<https://www.nhtsa.gov/sites/nhtsa.gov/files/documents/acrsfieldreference.pdf> (2017 Maryland State Police)

section 2.19 states:

"Distance Definition: The distance from the referenced Intersecting Road to the crash site. Explanation: The distance in feet or miles from the Intersecting Road to crash site."

Section 2.21:

"Distance Direction Definition: The compass direction describing the direction going from the primary and intersecting roads to the crash. Explanation: Units are described in compass points, i.e. North, South East and West. If the crash occurred in the intersection, select the compass point referenced in the Mile Point Direction box above."

▼ About Data

Crash Reporting - Incidents Data

Original data:

122,000 Rows

37 Columns

Each row is a "Collision"

https://data.montgomerycountymd.gov/Public-Safety/Crash-Reporting-Incidents-Data/bhju-22kf/about_data

"This dataset provides general information about each collision and details of all traffic collisions occurring on county and local roadways within Montgomery County, as collected via the Automated Crash Reporting System (ACRS) of the Maryland State Police, and reported by the Montgomery County Police, Gaithersburg Police, Rockville Police, or the Maryland-National Capital Park Police.

Please note that these collision reports are based on preliminary information supplied to the Police Department by the reporting parties. Therefore, the collision data available on this web page may reflect:

- Information not yet verified by further investigation
- Information that may include verified and unverified collision data
- Preliminary collision classifications may be changed at a later date based upon further investigation
- Information may include mechanical or human error"

"3.38 Going Direction Definition: The direction of the motor vehicles travel on the roadway prior to the crash. Explanation: This is not necessarily a compass direction, but must be consistent with the "Distance Direction" from the Log Mile Information for this roadway."

Montgomery County has committed to Vision Zero - to eliminating traffic fatalities and severe injuries by 2030. To achieve this ambitious goal, we must understand our crash history and the conditions that lead to crashes on our roadways. The Interactive Crash Map allows the public and county staff to analyze crash data for Montgomery County for crashes occurring between 2015 and now. This map is updated as soon as new data is made available.

Note: The Commission has conducted a thorough review of the County's data and has made revisions to that data in an effort to improve the locational accuracy of the crash data. The Commission's revised data is provided to the public for informational purposes only, and any reliance on the same is at the discretion and liability of the user. In the event a conflict exists between the Commission's data and the County's data, the County's data is the official record.

Start coding or generate with AI.

AI usage tracker:

- For all cells in merged_cleaned_df: if value in cell == "unkonwn" or "UNKNOWN" or "other" or "OTHER" or "Not Applicable" or "NOT APPLICABLE", set value to nan
- Compare values in each column.
If a value written in all caps has a matching value that is written in mixed case or lower case,
then convert all data written in all caps into the matching version that is written in mixed case or lower case.

```

# # Fix vehicle year from 0 to NaN
# # replaced selected, because used describe() to see minimum year
# merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(0, np
# merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(1, 200
# merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(2, 200
# merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(3, 200
# merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(4, 200
# merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(5, 200
# merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(8, 200
# merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(13, 20
# merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(14, 20
# merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(15, 20
# merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(97, 19
# merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(99, 19
# merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(198, :
# # BMW 328 per report was in production 1936–1940
# merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(200, :
# # Mazda CX–7 was in production 2006–2012
# merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(201, :
# # Chevy Suburban in production since 1934. impossible to extrapolate
# merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(202, :
# # temporarily change 215 to 2015 for
# merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(215, :
# merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(999, :
# #Mercedez–Benz C–class in production since 1993. Accident in 2024. :
# merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(1000, :
# #Toyota camry accident in 2016
# merged_df["Vehicle_Year"] = merged_df["Vehicle_Year"].replace(1005, :

# #TODO: re-check Lexus 2015

```


